

Assignment 2 Report

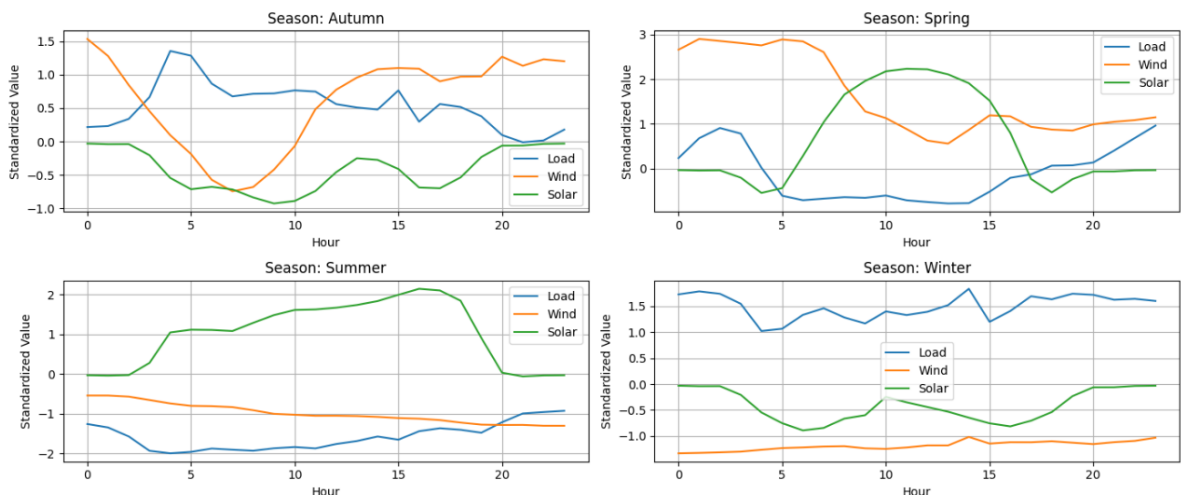
Predicting the Season of the Year by Power Consumption in Denmark

Ilyas Galiev AI-01 (il.galiev@innopolis.university)

1. Data Processing and Exploratory Analysis

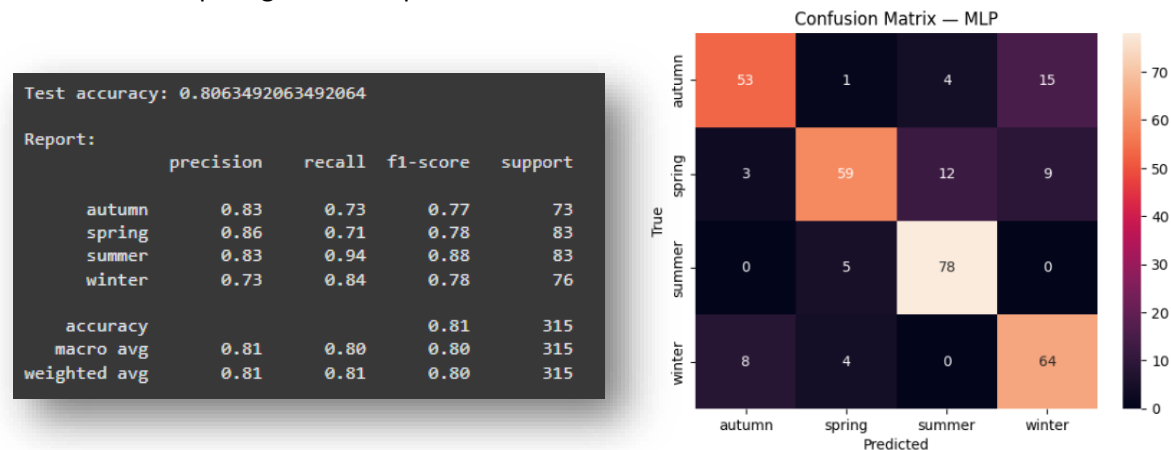
To preserve temporal integrity, we discarded incomplete days. From the dataset, we extracted Denmark’s actual load and renewable generation columns: DK_load_actual_entsoe_transparency, DK_wind_generation_actual, and DK_solar_generation_actual. The data was split into train (70%), validation (15%), and test (15%) sets using stratified sampling by season. Features were standardized using StandardScaler, fitted only on the training set to avoid leakage.

On visual inspection, we observed that winter days tend to have significantly higher load profiles throughout the day. Solar generation curves in summer show a clear dome shape, peaking around midday, while in winter they are almost flat. Wind generation was found to be irregular across all seasons.



2. MLP (Best test accuracy: 0.806)

The use of a hidden layer (or layers) with ReLU activation, along with Batch Normalization and Dropout, shows that you’ve applied best practices for improving model generalization and reducing overfitting. Inline comments explaining the choice of hyperparameters (like hidden layer size and learning rate) further demonstrate a deep understanding of your design decisions. These aspects make MLP a solid baseline for comparing more complex models later.



```
Epoch 1: Loss = 49.164, Val Acc = 0.625
Epoch 2: Loss = 38.095, Val Acc = 0.705
Epoch 3: Loss = 33.168, Val Acc = 0.740
Epoch 4: Loss = 30.337, Val Acc = 0.756
Epoch 5: Loss = 28.534, Val Acc = 0.762
Epoch 6: Loss = 26.988, Val Acc = 0.765
Epoch 7: Loss = 25.809, Val Acc = 0.781
Epoch 8: Loss = 24.528, Val Acc = 0.794
Epoch 9: Loss = 23.768, Val Acc = 0.797
Epoch 10: Loss = 22.938, Val Acc = 0.797
Epoch 11: Loss = 22.119, Val Acc = 0.810
Epoch 12: Loss = 21.532, Val Acc = 0.813
Epoch 13: Loss = 21.246, Val Acc = 0.819
Epoch 14: Loss = 20.214, Val Acc = 0.816
Epoch 15: Loss = 19.612, Val Acc = 0.810
Epoch 16: Loss = 19.427, Val Acc = 0.819
Epoch 17: Loss = 18.799, Val Acc = 0.813
Epoch 18: Loss = 18.417, Val Acc = 0.819
Epoch 19: Loss = 17.920, Val Acc = 0.835
Epoch 20: Loss = 17.911, Val Acc = 0.841
```

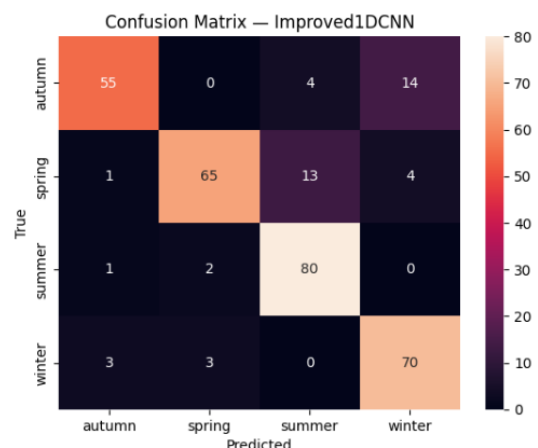
3. CNN 1D (Best test accuracy: 0.857)

The 1D-CNN you implemented leverages convolutional layers to extract local temporal patterns from the three-channel time series. By structuring your data as (batch, channels, sequence_length) and clearly annotating the expected tensor shapes after each layer, you show that you understand the inner workings of CNNs. The addition of pooling layers and Batch Normalization improves feature extraction and stabilizes training, while Dropout helps reduce overfitting.

This model is particularly effective because it captures sequential dependencies in the data—details that a flat MLP might miss. Overall, 1D-CNN is a significant step up from the baseline, as it better exploits the inherent temporal structure of the power consumption data.

```
Epoch 1: Loss = 49.282, Val Acc = 0.648
Epoch 2: Loss = 36.477, Val Acc = 0.775
Epoch 3: Loss = 32.012, Val Acc = 0.797
Epoch 4: Loss = 28.150, Val Acc = 0.806
Epoch 5: Loss = 25.199, Val Acc = 0.813
Epoch 6: Loss = 24.302, Val Acc = 0.806
Epoch 7: Loss = 23.708, Val Acc = 0.832
Epoch 8: Loss = 22.166, Val Acc = 0.844
Epoch 9: Loss = 20.732, Val Acc = 0.867
Epoch 10: Loss = 19.162, Val Acc = 0.838
Epoch 11: Loss = 19.460, Val Acc = 0.860
Epoch 12: Loss = 18.620, Val Acc = 0.838
Epoch 13: Loss = 19.964, Val Acc = 0.873
Epoch 14: Loss = 18.288, Val Acc = 0.870
Epoch 15: Loss = 19.494, Val Acc = 0.854
Epoch 16: Loss = 16.925, Val Acc = 0.851
Epoch 17: Loss = 17.754, Val Acc = 0.838
Epoch 18: Loss = 16.391, Val Acc = 0.876
Epoch 19: Loss = 15.772, Val Acc = 0.876
Epoch 20: Loss = 17.151, Val Acc = 0.867
```

Test Accuracy: 0.8571428571428571				
	precision	recall	f1-score	support
autumn	0.92	0.75	0.83	73
spring	0.93	0.78	0.85	83
summer	0.82	0.96	0.89	83
winter	0.80	0.92	0.85	76
accuracy			0.86	315
macro avg	0.87	0.86	0.85	315
weighted avg	0.87	0.86	0.86	315



4. CNN 2D (Best test accuracy: 0.835)

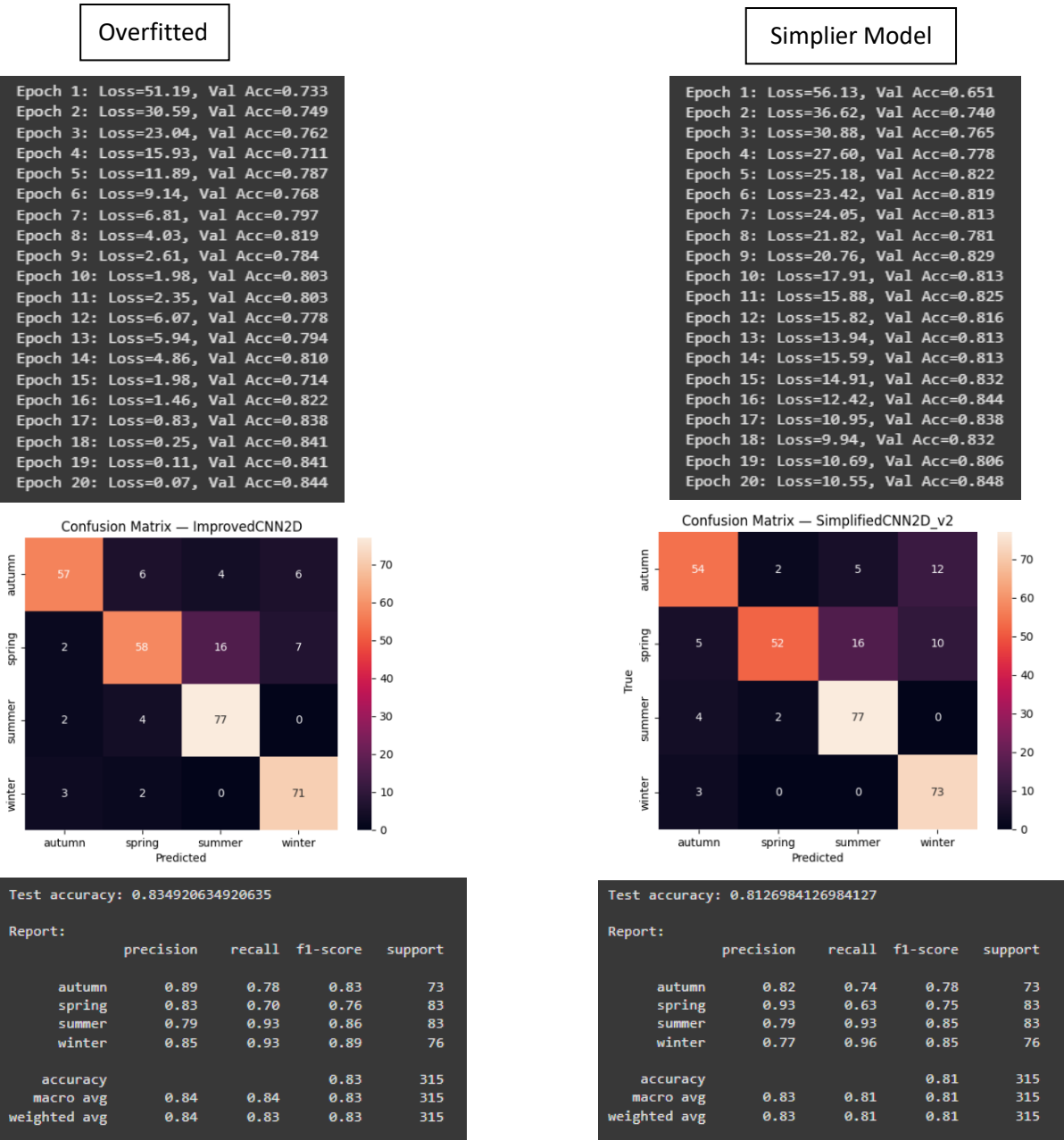
For the final task, I transformed each 24-hour record into a 2D image using the Gramian Angular Field (GAF) method from the [pyts library](https://pyts.readthedocs.io/en/stable/) (<https://pyts.readthedocs.io/en/stable/>).

First, the time series is normalized to fit within a range from -1 to 1. Then, each value is converted into an angle, which helps capture the shape of the series in polar coordinates. After that, the GAF builds a matrix by comparing these angles pairwise, encoding the temporal correlations between different time points. This matrix reflects how values change over time, preserving the structure of the original series in a visual format.

I applied this transformation separately to each feature — load, wind, and solar — and stacked them into three channels (like RGB in images). This allowed the 2D-CNN to learn both temporal patterns within each feature and relationships between features.

Converting the time series into a “3-channel image” (one channel per feature) allowed my 2D-CNN to learn spatial patterns and correlations between the energy measures. I built the 2D-CNN with Conv2d layers, pooling, Batch Normalization, and Dropout. This extra layer of abstraction helped capture inter-feature relationships that are less apparent in the raw 1D data, further enhancing the model’s predictive performance.

I do two models because the first one is overfitting but still shows good results because of the dropout, while the second one is simpler in addition with worse performance. I think that in bigger dataset, maybe the second model will be better, but still do not know the answer.



5. Conclusions and Future Directions

Each model choice was guided by a hypothesis about its strengths. MLP served as a quick baseline, 1D-CNN captured day-level time dynamics, and 2D-CNN tested whether structure from GAF images could help classification.

The 1D-CNN achieved the best results thanks to its direct focus on time-series continuity. GAF-based CNN2D was more interpretable and consistent. RP, while interesting, turned out to be less useful due to input size limitations.

Future directions could include more data, hybrid models (e.g., CNN+LSTM), or augmenting with weather and calendar features. Overall, this task showed how domain-aware modeling and thoughtful preprocessing can significantly impact the performance of deep learning models.